2024 20th International Conference on Synthesis, Modeling, Analysis and Simulation Methods and Applications to Circuit Design (SMACD) | 979-8-3503-5192-7/24/$31.00 ©2024 IEEE | DOI: 10.1109/SMACD61181.2024.10745428

Advanced Analog Design Optimization: Comparison Between Reinforcement Learning and Heuristic Algorithms

Michel Chevalier*, Severin Trochut*, Roberto Guizzetti*, Pascal Urard*,
Lioua Labrak†, John Samuel‡, Rémy Cellier† and Nacer Abouchi†
*STMicroelectronics, Crolles, France, michel.chevalier01@st.com
†Université de Lyon, CPE Lyon, Institut des Nanotechnologies de Lyon, Lyon, France, lioua.labrak@cpe.fr
‡Université de Lyon, CPE Lyon, LIRIS, Lyon, France, john.samuel@cpe.fr

***Abstract*—Last decades have seen a lot of research on Analog Design Automation. The most recent approaches are based on Reinforcement Learning (RL). This paper describes a new learning strategy enhancing the most recent Proximal Policy Optimization RL approach, applied to analog design. The method is compared to more classical heuristic approaches such as Ant Colony Optimization, Particle Swarm Optimization or Differential Evolution algorithm. This study is done using a gymnasium environment embedding a spice-like electrical simulator. The experiments are done under equivalent calculation conditions. The paper highlights convergence properties and demonstrates the RL ability to avoid local minimum traps.**

***Index Terms*—Reinforcement learning, Analog design sizing, Machine learning, Optimization**

I. Introduction

Both industry and scientific community have led decades of research in analog design optimization in order to simplify this complex time-consuming task. Analog sizing requires to fine-tune electrical components to meet electrical specifications (specs) and reach high performances that sometimes requires difficult trades-offs.

The introduction of machine learning techniques [1] allowed a breakthrough in the field. Using heuristic based algorithms such as Particle Swarm Optimization (PSO) [2], Ant Colony Optimization (ACO) [3], Differential Evolution (DE) [4], Grey Wolf Optimization (GWO) [5], or Evolution Strategy (ES) [6] was already a progress as design time was significantly reduced. However, new approaches based on artificial neural networks [7]–[9] have recently appeared. To compare them, this paper presents a Reinforcement Learning (RL) method and compare its efficiency with metaheuristic approaches on common benchmarks like a two-stage operational amplifier or a rail-to-rail input AB-class output buffer.

II. Analog Design Sizing

A. Context

Analog design sizing (ADS) is one of the key steps for a successful chip design. Starting from a schematic, the goal is to fulfill a set of N electrical specs chosen in a space S, by tuning the components parameters. From a specification in S, the problem is to find a correct set of parameters p in our design space P such that the ith electrical specification of our circuit $s_i(p)$ is meeting the ith goal specifications s_{gi}. The optimization problem presented in (1) represents the analog sizing challenge for a fixed topology.

$$\forall S_g \in S,\ \underset{p \in P}{\operatorname{argmin}} \sum_{i=0}^{N-1} s_i(p) - s_{gi} \tag{1}$$

Heuristics Methods (HM) have been already successfully used to tackle this problem [10]. However, [7], [8] proved the potential of RL applied to ADS.

III. Reinfocement Learning

A. Few Basic Concepts

RL [11] is a subfield of deep learning focused on training an agent to maximize a reward. In this approach, the agent interacts with an environment and learns from its experiences to maximize a cumulative reward. This trial and error process leads the agent to learn an optimal policy, enabling it to take the best decisions in a given context. In this study, the goal is to optimize the circuit's parameters to reach a given set of specs.

B. Environment

The gymnasium environment [12] embeds a SPICE-like simulator to observe the electrical circuit's behavior (Fig. 1). It computes the reward $r*$ (2), defined as the negative distance between the current specification and the goal.

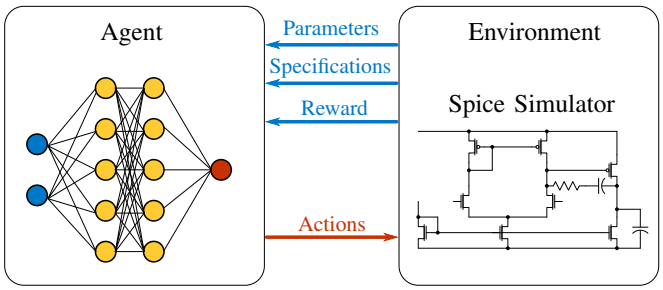


Fig. 1: Reinforcement learning interaction schematic, from [13].

Authorized licensed use limited to: Shanghai Jiaotong University. Downloaded on June 22,2026 at 18:04:14 UTC from IEEE Xplore. Restrictions apply.

$$r* = -\sum_{i \in I} \omega_i . min\left(\frac{goal_i - spec_i}{goal_i}, 0\right)$$
$$r = \begin{cases} r*, \text{ if } r* < 0 \\ R \end{cases} \quad (2)$$

R represents a positive bonus given when the specification is met. ω_i is the $spec_i$'s priority weight. The reward function r is designed such that the bonus is always greater than the negative reward accumulated during the episode. Thus, the agent is still able to achieve the goal even after a series of penalties that could deviate it from the desired specs.

C. Agent

The agent is composed of two elements: the actor and the critic [11]. Both embed a fully connected neural network with two hidden layers (400 and 300 neurons respectively Fig. 1) that takes as input the current circuit parameters (width, length, number of transistors in parallel, capacitor and resistor values) and the electrical specs (consumption, phase margin, etc.). The actor changes the circuit parameters while the critic approximates the cumulative reward. This approximation is used as a loss in gradient descent algorithms to update the actor's weights.

D. Trainer and Learning Strategy

As demonstrated in [7] and [13], the Proximal Policy Optimization (PPO) algorithm [14] was used to train the agent. This process was enhanced by a new learning strategy (Fig. 2) called *Incremental Learning Strategy* (ILS), inspired by [15].

Starting close from several goal points, the agent is forced to start its trajectory farther and farther from the goals. At a certain time in the training, the different zones overlap (as illustrated by light orange zones in Fig. 2) and the training becomes comparable to [13].

ILS goal is to improve the agent's convergence by exploring simple problems and then increasing their complexity.

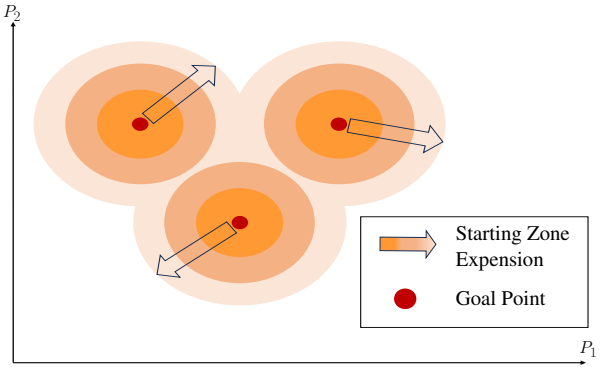


Fig. 2: Incremental Learning Strategy illustration for an agent learning in a two dimensional space.

IV. Experiments and Results

A. Benchmarks

1) Two Stage Operational Amplifier: The first benchmark used (Fig. 3) is a CMOS two-stage simple operational amplifier (Simp. Op. Amp.) commonly used in the literature [7] [16]. The topology embeds 11 devices and is desbribed by 14 parameters and 8 specs (plus transistor biasing constraints) as shown in Tab. I.

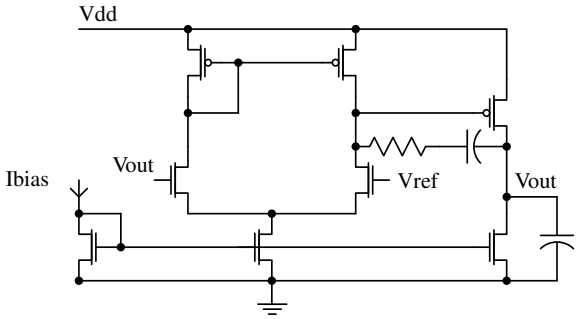


Fig. 3: Two-stage operational amplifier.

2) Rail-to-Rail AB-Class Output Buffer: The second benchmark used is a CMOS Rail-to-Rail AB-class output buffer (Rail-to-Rail) used by [13]. This amplifier, involves 32 components and is described by 38 parameters and 8 specs as shown in Tab. I, and presented in 4.

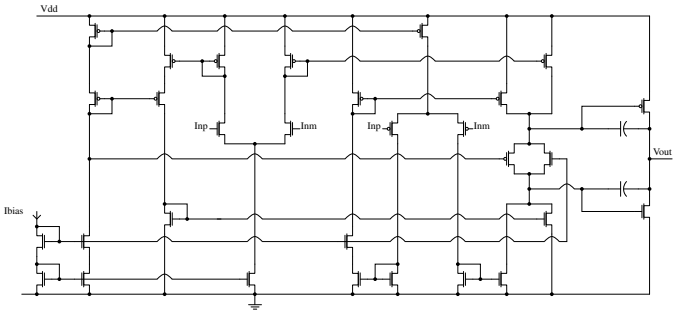


Fig. 4: Rail-to-rail input opamp.

B. Reinforcement Learning : Training

During the training, agents' weights were saved every 100,000 steps, to compare the ILS strategy and the standard PPO training, referred as Naive Learning (NL). For each benchmark, the agents are tested on 200 sets of specifications (inference dataset) different from the training dataset. As the trajectory results rely on the starting point, agents were evaluated on a population of 60 individuals with random starting points. Each individual made a trajectory independent of the others. Computation was done on 60 CPU cores. Fig. 5 represents the inference results at different training steps: every count of 100,000 simulations for the blue curves, and every count of 1,000,000 simulations for the red curve.

The dashed plot represents the result of the NL in inference. The differences between the two blue plots highlight the

Authorized licensed use limited to: Shanghai Jiaotong University. Downloaded on June 22,2026 at 18:04:14 UTC from IEEE Xplore. Restrictions apply.

TABLE I: Benchmarks Specifications

Circuit	Rail-to-Rail	Simp. Op. Amp.
Capacitance (pf) $[min, max]$ $incr = 0.1$	$[15, 17]$	$[0.5, 10]$
Resistance ($k\Omega$) $[min, max]$ $incr = 1$	-	$[10, 100]$
Width (µm) $[min, max]$ $incr = 0.1$	$[0.4, 26]$	$[0.4, 40]$
Length (µm) $[min, max]$ $incr = 0.1$	$[0.7, 10]$	[0.5, 10]
Multiplier $[min, max]$ $incr = 1$	$[1, 35]$	$[1, 50]$
Sampling Space $[min, max]$	PHASE MARGIN $\in [50, 70](°)$, GAIN MARGIN $\in [10, 20](dB)$ PSRR MAX $\in [-18, -5](dB)$, DC GAIN $\in [60, 95](dB)$ UGF $\in [8 \times 10^4, 1.6 \times 10^5](Hz)$, CONSO $\in [4 \times 10^{-5}, 11 \times 10^{-4}]$ (A) ABS OFFSET $\in [2 \times 10^{-4}, 3 \times 10^{-3}](V)$ GAIN PEAKING $\in [-10^{-7}, -10^{-9}](dB)$	PHASE MARGIN $\in [50, 90](°)$, GAIN MARGIN $\in [10, 35](dB)$ PSRR MAX $\in [-25, -5](dB)$, DC GAIN $\in [70, 100](dB)$ UGF $\in [5 \times 10^6, 15 \times 10^6](Hz)$, CONSO $\in [10^{-5}, 2 \times 10^{-5}](A)$, ABS OFFSET $\in [2 \times 10^{-4}, 3 \times 10^{-3}](V)$ GAIN PEAKING $\in [-10^{-7}, -10^{-9}](dB)$

effectiveness of the ILS approach compared to the NL strategy in finding the 200 specs at different phases of the learning. The poor results of the NL strategy are due to a need of ahuge coverage of the parameters' space to find solutions.

In contrast, the ILS approach learns how to access working zones and finds interesting parameters. It is able to find 43% of the 200 goals with only 100,000 simulations. ILS is able to increase the percentage of goals found to 72% by the end of the training (refer to red plot in Fig. 5). ILS produces accurate results very quickly making it a highly effective strategy of learning.

C. Reinforcement Learning : Inference

After the training phase, the agent's ability to reach different targets has been challenged. Tested on the previous datasets, Fig. 6 presents a review of the different behaviors observed in inference for the ILS agent on the Rail-to-Rail. The experiment has been realized using 1 CPU core to review the agent's trajectories from a common starting point.

Fig. 6 shows a specific behavior (blue curve), where a small first plateau has been reached. This corresponds to a local minimum avoided by the agent by jumping further to finally reach a solution. To compare, the red curve shows an optimization process where the agent does not encounter local minima. Two trajectories in orange and green illustrate the

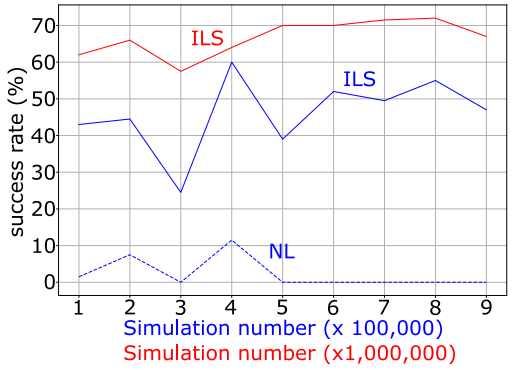


Fig. 5: Inference success rate of the agents during their training on Simp. Op. Amp.

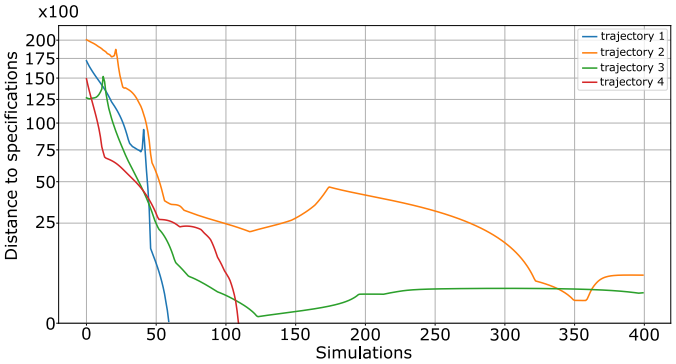


Fig. 6: Different Inference Behaviors

cases where the specs goals were not feasible. This highlights the jumps from a local minimum to another to explore the space, without getting trapped inside of them.

D. Reinforcement Learning and Heuristic Methods Comparison

Based on the two previous benchmarks, the ILS agent was compared in inference to population-based heuristic methods and NL agent. Those HM have been selected for mimicking "natural agents" behaviors. To build a fair comparison, ILS agent and NL agent has been trained for the same amount of steps: 1,000,000, and in inference, the principles of the HM have been mimicked. At first, 60 random starts have been generated. Then, one RL agent has been deployed for each start, reiterated for each specification goal in the inference dataset. In the same way, HM were initialized with 60-individual populations, placed at the same random starting points. The size of the population was fixed to 60 according a tuning of this hyperparameter to find a correct trade off between the optimization time and the accuracy of the HM algorithms.

As shown in Tab. II, GWO gives the best results on Simp. Op. Amp., in terms of the number of iterations (iter) and the number of solutions found. However, RL_{ILS} has still some strong results, confirming the improvements of ILS regarding NL, and achieving to excel three HM methods. On the second

Authorized licensed use limited to: Shanghai Jiaotong University. Downloaded on June 22,2026 at 18:04:14 UTC from IEEE Xplore. Restrictions apply.

TABLE II: Mean iterations and percentage of achieved spec in the inference dataset

Method	PSO		GWO		DE		ES		ACO		RL_{ILS}		RL_{NL}	
	Iter	Score	Iter	Score	Iter	Score	Iter	Score	Iter	Score	Iter	Score	Iter	Score
Simp. Op. Amp	43.55	76.0%	**25.35**	**79.5%**	67.3	70.0%	68.75	53.0%	78.75	26.0%	62.96	74.0%	436.16	2%
Rail-to-Rail	50	86%	25.5	100%	57	38%	68.75	86%	60.5	12%	**7.72**	**100%**	427.3	7%

TABLE III: Mean optimization time comparison.

Method	PSO	GWO	DE	ES	ACO	RL_{ILS}	RL_{NL}
Simp. Op. Amp.	**18.1s**	10.5s	27.9s	28.6s	32.7s	26.1s	181.2s
Rail-to-Rail	45s	22.95s	51.3s	45.9s	46s	**6.9s**	381.9s

benchmark, GWO is still the best HM method in terms of number of iterations, and found all the specifications' sets. However, the RL_{ILS} agent found the same amout of specs (Tab. II), but faster than GWO (Tab II and Tab III), becoming the best method between HM and RL on this benchmark. According to the different time results (Tab III), it appears that the optimization time is proportionnal by a factor 1.11 to the number of iterations on the Rail-to-Rail, respectively 0.42 for the Simp. Op. Amp. This highlights the time of optimization is mostly spent in the SPICE simulations, regardless of the method used.

V. Discussion

A first comparison of the training between NL and ILS proved the efficiency of the in incremental learning strategy to improve the agent's convergence. Combining state of the art agents [13], [17], [18] with ILS should improve the results on the Simp. Op. Amp. in inference. Then, inference quality of the ILS agent has been clearly demonstrated regarding to the NL agent.

As metaheuristics methods have been designed to avoid local minima, the ILS agent's behavior in inference should also avoid this traps. This is a key step in applying RL to ADS. Even if the trained agent is performing well on the two benchmarks, the real number of simulations to compare with HM should have been included with the training ones. Even if it only takes 7.72 iterations to optimize the Rail to Rail, it required a million of steps to train the agent. Using state-of-the-art agents might reduce the amount of simulations needed to train and achieve similar results.

Last but not least, the experiment highlights the ability of the RL to tackle high dimension problem, faster than HM.

VI. Conclusion

The goal of this study was to present a fair comparison between heuristic methods and reinforcement learning on two benchmarks for analog design optimization. First the efficiency of the incremental learning strategy has been exposed. Then, the agent's ability to avoid local minima in inference has been illustrated on fully described benchmarks. Finally, the comparison between RL agents and HM shown the RL ability to tackle high complexity designs better than metaheuristics methods, even if those two approaches look comparable on simple circuits. Those results could be enhanced by a further study including variations of process, temperature and voltage in the circuit's optimization.

VII. Acknowledgment

The authors would like to thank A. Vialle, J. Coulon, Z. Blanchin, T. Pincemin, G. Huth, M. Khader, F. Pautet, M. Aymes, N. Chanez.

References

[1] G. Huang et al. "Machine learning for electronic design automation: A survey", 2021.
[2] Gad, Ahmed G, "Particle swarm optimization algorithm and its applications: a systematic review", 2022, 29, 5, 2531–2561, Archives of computational methods in engineering, Springer
[3] M. Dorigo, V. Maniezzo, and A. Colorni, "Ant system: optimization by a colony of cooperating agents", 1996, 26, 1, 29–41, IEEE transactions on systems, man, and cybernetics, part b (cybernetics), IEEE
[4] Das, Swagatam and Mullick, Sankha Subhra and Suganthan, Ponnuthurai N, "Recent advances in differential evolution–an updated survey", 2016, 25, 1–30, Swarm and evolutionary computation, Elsevier
[5] Majeed, MA Mushahhid and Patri, Sreehari Rao, "An enhanced grey wolf optimization algorithm with improved exploration ability for analog circuit design automation", 2018, 26, 5, 2605–2617, Turkish Journal of Electrical Engineering and Computer Sciences
[6] A. lowik, H. Kwasnicka, Halina,"Evolutionary algorithms and their applications to engineering problems"S, 2020, 32, 12363–12379, Neural Computing and Applications, Springer
[7] K. Settaluri, A. Haj-Ali, Q. Huang, K. Hakhamaneshi and B. Nikolic, "AutoCkt: Deep Reinforcement Learning of Analog Circuit Designs," 2020 Design, Automation & Test in Europe Conference & Exhibition (DATE), 2020
[8] N. Lourenço et al., "Using polynomial regression and artificial neural networks for reusable analog IC sizing",2019,13–16, 2019, 16th International Conference on Synthesis, Modeling, Analysis and Simulation Methods and Applications to Circuit Design (SMACD), IEEE
[9] Mina, Jabbour, Sakr, "A Review of Machine Learning Techniques in Analog Integrated Circuit Design Automation. Electronics" 2022
[10] Talbi, El-Ghazali and Basseur, Matthieu and Nebro, Antonio J and Alba, Enrique,"Multi-objective optimization using metaheuristics: non-standard algorithms",2012, 19, 1–2, 283–305, International Transactions in Operational Research, Wiley Online Library
[11] Sutton, Richard S and Barto, Andrew G, "Reinforcement learning: An introduction", 2018,MIT press.
[12] M. Towers et al., "Gymnasium", 2023, Zenodo
[13] M. Chevalier et al., "Reinforcement Learning for Analog Sizing Optimization", 2023, 1–4, 2023 19th International Conference on Synthesis, Modeling, Analysis and Simulation Methods and Applications to Circuit Design (SMACD), IEEE
[14] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms", 2017, arXiv preprint arXiv:1707.06347
[15] Y. Bengio, J. Louradour, R. Collobert, and J. Weston. 2009. Curriculum learning. In Proceedings of the 26th Annual International Conference on Machine Learning (ICML '09). ACM, New York, NY, USA, 41–48.
[16] Y. Uhlmann, M. Brunner, L. Bramlage, J. Scheible, and C. Curio, "Procedural-and Reinforcement-Learning-Based Automation Methods for Analog Integrated Circuit Sizing in the Electrical Design Space", 2023, 12, 2, 302, MDPI
[17] Cao, Benosman, Zhang, Ma, "Domain Knowledge-Infused Deep Learning for Automated Analog/Radio-Frequency Circuit Parameter Optimization", arXiv, 2022
[18] H. Wang et al., "GCN-RL Circuit Designer: Transferable Transistor Sizing with Graph Neural Networks and Reinforcement Learning", The 57th Design Automation Conference (DAC), 2020

Authorized licensed use limited to: Shanghai Jiaotong University. Downloaded on June 22,2026 at 18:04:14 UTC from IEEE Xplore. Restrictions apply.